Ablation Protocol: Do Pre-Generated Conversion Guides Reduce Silent Translation Error?

July 19, 2026

1 Purpose and Scope

The r2py methodology's central methodological claim is that generating a machine-readable translation guide for every language construct *before* any function is converted reduces silent numerical error and enforces consistency across independent conversion-agent invocations. Both technical reports state this claim explicitly and motivate it by the stochastic nature of LLM inference. Neither report tests it.

At present the evidence for the claim is a single anecdote: in `KernSmooth` Phase 4, the conversion agent for `linbin` subscripted the `None` return of an f2py-wrapped subroutine while the agent for `rlbin`—processing structurally near-identical code—correctly used in-place output semantics. The divergence is attributed to the two invocations being independent. This is a compelling illustration, but it is one observation, and it does not establish that the guides are what prevents such divergence.

This protocol specifies a scoped experiment that converts that anecdote into a measurement. It is deliberately *not* a competitive benchmark against external code-translation systems (TransCoder, general LLM translation baselines); that is the evidence bar of a software-engineering conference and is out of scope here. It is an ablation of the methodology's own components, designed to answer one question: *if the guides are removed and everything else is held constant, does translation quality degrade, and by how much?*

Design principle. Every element below is chosen so that a skeptical reviewer cannot attribute a positive result to experimenter latitude. The conditions differ by exactly one manipulable factor, grading is fully automated against a pre-existing oracle, the analysis plan is fixed before data collection, and the largest threat to validity (training-data contamination) is both measured and shown to bias against the hypothesis.

2 Hypotheses

The claim decomposes into two logically distinct sub-claims that require different measurements. Stating them separately matters: it is possible for the guides to improve consistency without improving correctness, or the reverse, and each outcome carries a different message for the paper.

H1 (Correctness). Functions converted with access to the language-dependency conversion guides

pass the R-benchmarked reference test suite at a higher rate than functions converted without them.

H2 (Consistency). Within a single conversion run, a given R language construct is translated to a smaller number of distinct Python idioms when guides are supplied than when they are not.

H3 (Error character, exploratory). Failures in the no-guide condition are disproportionately *silent* (numerically wrong output, no exception raised) relative to failures in the guide condition, which are expected to skew toward loud failures (exceptions, crashes).

H3 is the most consequential for the paper's framing if it holds, because the methodology's stated motivation is specifically the danger of silent numerical error rather than of failures that announce themselves. It is labelled exploratory because the expected failure counts may be too small to support a powered test; it will be reported descriptively if so.

Pre-specified null results. If H1 fails to reach significance but H2 holds, the paper's claim should be narrowed to consistency and maintainability rather than correctness, and stated that way. If both fail, that is a publishable negative result about the cost-effectiveness of the guide phases and should be reported as such rather than suppressed. This commitment is recorded here, before data collection, deliberately.

3 Experimental Design

3.1 Conditions

Three conditions, differing only in the context supplied to the `@convert-r-function-to-python` sub-agent.

Condition	Context supplied	Role
G (guides)	R source fragment, structural dependency JSON, *full conversion-guide set*, converted callee artifacts	Treatment. Reproduces the published pipeline exactly.
N (no guides)	R source fragment, structural dependency JSON, converted callee artifacts. Guide-injection block removed; *no other change*.	Control. Isolates the guides as the manipulated factor.
P (placebo)	As N, plus a generic, construct-agnostic instruction block of comparable token length (e.g. "translate R idioms to idiomatic NumPy; be careful with indexing, types, and normalisation conventions").	Controls for context volume. Distinguishes "the guides help" from "more context helps."

Condition P is what separates this experiment from a weak ablation. Without it, a reviewer can object that the treatment agent simply received more tokens of instruction and that any generic prompt of similar length would have produced the same benefit. P is the more expensive condition to justify but the one that makes the result defensible. **If budget forces a reduction, run G and N at full replication first and P at reduced replication on a function subset**, reporting it as a secondary check rather than dropping it.

Prompt construction discipline. The N prompt must be produced from the G prompt by deleting the guide-injection block and nothing else—no rewording, no compensating instructions, no removal of adjacent text. Both prompts are to be committed to the repository *before* any run, and their diff included in the paper's supplementary material. This forecloses the objection that the control was handicapped.

3.2 Unit of Analysis and the Substitution Harness

The unit is a single **(function, condition, replicate)** triple.

Evaluating a converted function requires the rest of the package to be correct, so conversions are not assembled into a whole package. Instead:

1. Begin from the released, fully validated package (`r2py_kernsmooth` at 518/518, or `r2py_rpart` at 844/846) as a known-good substrate.
2. Convert exactly one function f under condition c, replicate r.
3. Splice the resulting function into the known-good package, replacing the released implementation of f and nothing else.
4. Run the subset of the reference test suite that exercises f.
5. Record the outcome, then restore the released implementation before the next trial.

This single-function substitution design isolates the treatment effect per function and prevents an early bad conversion from cascading into downstream failures that would be misattributed to the condition.

Test-subset determination. For each f, the exercising test subset is computed once, in advance, by mutation: deliberately corrupt the released f (e.g. perturb a return value) and record which tests fail. That set is fixed and reused across all conditions and replicates for f. Any function whose exercising subset is empty is excluded from the experiment and the exclusion reported—an untested function cannot contribute evidence either way.

3.3 Function Sample

Tier	Content
Primary	All `KernSmooth` functions with a non-empty exercising test subset (expected: most of the 16, certainly the 7 public ones). Cheap, fast, backed by the 518-test oracle.
Confirmatory	A stratified sample of `rpart` functions, drawn to over-represent those using the constructs the reports flag as high-risk (`as.integer`, `unlist`, `tapply`, `t`, `format`, `switch`, `missing`). Target 12–15 functions.

Running both tiers is what lets the paper claim the effect is not an artifact of one package's characteristics. The `rpart` tier is also where the harder constructs live, so it is where the effect, if real, should be largest. A pre-registered prediction to that effect strengthens the result if borne out.

3.4 Replication and Power

LLM inference is stochastic; a single run per cell measures noise, not effect. Since stochasticity is the methodology's own stated motivation for the guide phases, the experiment must sample the stochastic distribution rather than draw one point from it.

Pilot first. Before committing to the full matrix, run a pilot: 5 `KernSmooth` functions × conditions G and N × 3 replicates (30 invocations). Its sole purpose is to estimate the between-replicate variance in pass rate, which is currently unknown and which determines the replication needed. Do not draw inferential conclusions from the pilot.

Planned replication. Absent pilot data, plan on $R = 10$ replicates per cell. For the primary tier this is roughly $16 \times 3 \times 10 = 480$ invocations; for the confirmatory tier roughly $15 \times 3 \times 10 = 450$. Both are well within the scale of agent dispatch already routine in this project (Phase 8.2 of `rpart` dispatched 172 agents in 9 batches). Adjust R upward if the pilot shows high variance.

Temperature and model pinning. Record the model identifier and any sampling parameters, and hold them fixed for the entire experiment. If the model version changes mid-experiment, the affected cells must be re-run; a version change is a confound, not a nuisance. Log the model string with every trial.

4 Outcome Measures

4.1 Primary: Correctness Rate (H1)

Binary per trial: does the spliced package pass *all* tests in f's exercising subset?

$$\text{pass}(f, c, r) \in \{0, 1\}$$

Grading is fully automated via `pytest` against the existing R-benchmarked oracle. No human judgement enters the primary outcome, which removes grader bias entirely.

4.2 Secondary: Failure Taxonomy (H3)

Every failing trial is classified automatically into one of:

Silent numerical Tests fail on value assertions; no exception raised. *The category the methodology exists to prevent.*

Loud An exception, crash, or import error.

Structural Output shape, type, or container mismatch.

Classification is by inspection of the pytest failure mode (assertion versus error), so it too is mechanical.

4.3 Mechanism: Idiom Dispersion (H2)

For each language construct k and each complete run, count the number of *distinct normalised Python idioms* used to translate k across all functions in that run:

$$D_k = \left|\{\, \text{normalise(translation of } k \text{ at site } s) : s \in \text{sites}(k) \,\}\right|$$

Normalisation strips whitespace, comments, and local variable names, retaining the called API and argument structure; the normaliser must be written and frozen *before* data collection, and applied blind to condition. $D_k = 1$ means the construct was translated identically everywhere.

Report mean D_k over constructs with ≥ 3 call sites, by condition. This is the direct operationalisation of the consistency claim and, to our knowledge, is not a metric used in prior code-translation work—it is a modest methodological contribution in its own right and worth naming as such in the paper.

5 Contamination: The Principal Threat

`r2py_kernsmooth` and `r2py_rpart` are public on GitHub and PyPI, and the upstream R packages have been public for decades. A model may therefore reproduce a known-correct translation from memory rather than derive it from the supplied guides. This is the most serious threat to validity and must be addressed directly rather than acknowledged in passing.

It biases toward the null. Contamination inflates the *control* condition's performance, since the no-guide agent can fall back on memorised answers. The measured effect of the guides is therefore a *lower bound* on their true effect. A statistically significant positive result is conservative under contamination. This argument should appear explicitly in the paper; it converts the threat from a weakness into a strengthening qualifier.

Memorisation probe. Run a cheap direct test: prompt the agent for the Python translation of a named function with *no* R source supplied, and measure how closely the output matches the released implementation. High similarity is direct evidence of memorisation and quantifies the contamination floor. Run this for every function in the sample; report the distribution.

Held-out package (strongest control). The definitive answer is to run a reduced version of the experiment on a package this project has never converted and never published. `acepack` and `logspline`—both named as generality-test candidates in the May 11 addendum—are the natural candidates. Even a small held-out arm (5–8 functions, G and N only) removes the contamination objection for that arm entirely, and it simultaneously supplies the third-package generality data point the venue analysis contemplated. **Recommended if schedule permits; this is the single highest-value optional addition in this protocol.**

6 Randomisation, Blinding, and Pre-Registration

Randomised order. Trials are executed in randomised order over (function, condition, replicate), not grouped by condition, so that any drift in model behaviour over the experiment window distributes evenly across conditions rather than confounding with one.

Blind analysis. Outcomes are recorded against opaque condition labels. The mapping from label to condition is not revealed until the analysis script is written and frozen. Since grading is automated this is straightforward to enforce and cheap to claim honestly.

Pre-registration. Before the first non-pilot trial, commit to the repository: the two prompt variants and their diff, the idiom normaliser, the per-function test subsets, the analysis script, and this protocol. Tag the commit. Reviewers at selective venues respond well to a timestamped pre-registration, and it costs essentially nothing here.

7 Analysis Plan

Fixed in advance; deviations to be reported as such.

H1. Mixed-effects logistic regression on the binary pass outcome, with condition as a fixed effect and function as a random intercept:

$$\operatorname{logit}\big(\Pr[\text{pass}]\big) = \beta_0 + \beta_1 \mathbf{1}[c = G] + \beta_2 \mathbf{1}[c = P] + u_f$$

The random intercept accounts for functions differing intrinsically in difficulty; without it, replicates within a function would be treated as independent, overstating precision. Report odds ratios with 95% confidence intervals and the marginal difference in pass rate. A simpler paired Wilcoxon signed-rank test on per-function pass proportions should be reported alongside as a distribution-free robustness check.

H2. Paired comparison of mean D_k between conditions, paired by construct, via Wilcoxon signed-rank. Report the effect size.

H3. Report the failure taxonomy as a contingency table by condition, with Fisher's exact test if expected cell counts permit; otherwise descriptive only, as pre-specified.

Reporting discipline. Report effect sizes with confidence intervals as the primary quantitative result. Do not report p-values without accompanying effect sizes. Report all pre-registered outcomes including null ones.

8 Threats to Validity

Contamination. Addressed in Section 5. Biases toward the null; quantified by the memorisation probe; eliminated in the held-out arm if run.

Oracle incompleteness. The test suites, though extensive, cannot detect a silent error on a code path they do not exercise. Measured pass rates are therefore upper bounds on true correctness in *both* conditions. This inflates both arms and should not bias the comparison, but it bounds the absolute claim and must be stated.

Single-function isolation. The substitution harness deliberately removes the compounding of errors across functions. Real conversion runs convert everything, so the harness likely *understates* the practical benefit of guides, since it eliminates the cross-function inconsistency the guides are partly designed to prevent. H2 is measured on full runs precisely to recover this.

Single model family. Results are established for one model. The methodology is claimed to be tool-agnostic; the experiment does not establish that. State the limitation, and if cheap, run a reduced arm on a second model family as a generality check.

Construct-frequency imbalance. Constructs used at only one or two sites cannot exhibit dispersion; the ≥ 3 site threshold for H2 is pre-specified to avoid post-hoc threshold selection.

9 Execution Sequence

1. Build the substitution harness and the per-function test-subset map (via mutation). Freeze.
2. Write and freeze the two prompt variants, the placebo block, and the idiom normaliser. Commit and tag.
3. Run the memorisation probe; record the contamination floor.
4. Run the pilot (30 invocations). Estimate variance; set R.
5. Pre-register the full protocol and analysis script. Tag.
6. Execute the primary tier (`KernSmooth`) in randomised order.
7. Execute the confirmatory tier (`rpart`).
8. Optional, recommended: execute the held-out arm (`acepack` or `logspline`).
9. Unblind, run the frozen analysis, report all pre-registered outcomes.

Steps 1–5 are the substantive work and are largely mechanical engineering against infrastructure that already exists. Steps 6–8 are compute time rather than person time.

10 What This Contributes to the Manuscript

The experiment supplies the manuscript with:

- A quantified answer to "do the guide phases earn their cost?"— which is the first question a referee will ask about a seven-phase pipeline in which two phases exist solely to produce documentation consumed by other agents.
- An effect size for the paper's central claim, replacing the current single anecdote.
- The idiom-dispersion metric, which operationalises "consistency" in a way prior code-translation work (judged on functional or structural equivalence) does not.
- If H3 holds, direct evidence for the specific claim that the methodology suppresses *silent* error, which is the framing that distinguishes this work from general code translation.
- A pre-registered design, which is unusual in this literature and is cheap credibility at a selective venue.

A null result remains publishable and should be reported: it would indicate that modern models handle these constructs adequately without explicit guidance, that the methodology's value lies

in auditability and consistency rather than raw correctness, and that the guide phases could be simplified in future conversions. That is a useful finding for anyone adopting the framework, and committing to report it in advance is what makes the positive result credible if it arrives.

End of protocol.